EGYPT-JAPAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Computer Science and Engineering

CSE 433 — Deep Learning

Used Car Price Prediction

Using Deep Learning

Technical Report — Final Submission

Submitted to:

Assoc. Prof. Dr. Mohammed Kamal Abdel Salam

Team Members:

Jana Elsalhy	120220093
Shiref Ashraf	120220099
Ahmed Nagah	120220116
Tasbih Othman	120220117
Karim Mazroua	120220118
Yasmeen Sameh	120220143

Submission Date: May 14, 2026

Discussion Date: May 14, 2026

Contents

Abstract

This report presents the design, implementation, and evaluation of a deep learning regression system for predicting used car prices. The system addresses a practical challenge in the automotive marketplace: estimating fair market value from a structured set of vehicle attributes. A fully connected neural network was trained on approximately 3,000 real-world-style used car listings and achieved an R^2 of 0.9899, a Mean Absolute Error (MAE) of \$651.32, and a Mean Absolute Percentage Error (MAPE) of 5.75% on a held-out test set. The pipeline incorporates log-transformation of the target variable, leakage-free encoding, outlier clipping, batch normalisation, dropout regularisation, early stopping, and adaptive learning-rate scheduling. A live Flask web application and a command-line inference interface complete the end-to-end deployment.

1. Problem Definition

1.1 Motivation and Context

The global used car market is characterised by severe information asymmetry: buyers cannot reliably assess whether a listed price is fair, and sellers lack objective benchmarks to price their vehicles competitively. Human-based appraisal methods are inconsistent, slow, and subject to cognitive bias. Automated pricing models, powered by machine learning, offer a scalable and data-driven alternative that benefits every participant in the ecosystem — buyers, sellers, dealerships, insurers, and online platforms.

1.2 Formal Problem Statement

Let $\mathbf{x} \in \mathbb{R}^n$ be a feature vector describing a used vehicle (brand, model, year, engine displacement, power, fuel type, transmission, ownership history, and mileage). The objective is to learn a regression function $f : \mathbf{x} \mapsto y$, where $y \in \mathbb{R}$ represents the vehicle's selling price in USD, such that a chosen loss metric (MSE in log-space) is minimised on unseen test data.

1.3 Objectives

- Train a deep neural network regressor that achieves $R^2 \geq 0.95$ on the test set.
- Build a preprocessing pipeline that is provably free of data leakage.
- Quantify feature importance to identify the dominant pricing drivers.

- Deploy an interactive web application for real-time price estimation.
- Analyse error patterns and identify systematic failure modes.

2. Literature Review

Vehicle price prediction has attracted substantial research attention over the past decade. The following papers are directly relevant to the methodology adopted in this project.

2.1 Gradient Boosting vs. Neural Networks for Tabular Data

Prokhorenkova et al. (2018). "CatBoost: Unbiased Boosting with Categorical Features." NeurIPS.

This seminal paper introduced CatBoost, a gradient-boosted decision-tree algorithm specifically designed to handle high-cardinality categorical features — such as car brand and model — without the data leakage that plagues naive one-hot encoding. The authors demonstrate that ordered target encoding (computing category statistics on a subset of examples that precede each training sample) eliminates the shift between training and test distributions. Our pipeline adopts a related leakage-free target encoding strategy for the `Model` column: target statistics are computed exclusively on the training split before being mapped to both train and test sets. The paper also benchmarks tabular datasets and shows that well-tuned gradient boosting can outperform deep networks on small to medium datasets ($<$ 100K samples), a finding that motivates our Limitations discussion.

2.2 Deep Neural Networks for Car Price Regression

Pudaruth (2014). "Predicting the Price of Used Cars Using Machine Learning Techniques." International Journal of Information & Computation Technology.

This early study benchmarked multiple regression algorithms — linear regression, kNN, Naive Bayes, and decision trees — on a small used-car dataset. Numeric features (age, mileage, engine size) were found to carry the most predictive power, while brand and model required careful encoding. Although the dataset and models are modest by modern standards, the paper established core findings that remain valid: vehicle age and mileage are the dominant depreciation signals, and tree-based methods outperformed linear models on non-linear price relationships. Our work builds on this foundation by substituting the shallow models with a four-layer feed-forward network and introducing log-transformation of the price target to handle right skewness — a limitation the original study acknowledged but did not address.

2.3 Regularisation via Batch Normalisation

Ioffe and Szegedy (2015). "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." ICML.

Batch Normalisation (BN) standardises layer inputs across each mini-batch during training, reducing internal covariate shift and allowing higher learning rates. In regression tasks on tabular data, BN has been shown to stabilise loss trajectories and reduce sensitivity to weight initialisation. Our model inserts BN layers after the first two Dense layers (128 and 64 neurons), which produced smoother training curves and enabled faster convergence. The paper's analysis of reduced gradient vanishing is particularly relevant because our network predicts log-space prices spanning a wide numeric range; without BN, the deeper layers receive poorly scaled gradients.

2.4 Hyperparameter Optimisation

Bergstra and Bengio (2012). "Random Search for Hyper-Parameter Optimization." JMLR 13.

This paper demonstrates that random search over hyperparameter spaces outperforms manual grid search in both efficiency and final performance, because not all hyperparameters affect model performance equally. For our project, hyperparameters including number of layers, width, dropout rates, and learning rate were manually tuned. The paper provides theoretical grounding for the observation that varying dropout rate and learning rate had a larger impact on validation loss than varying layer width. Future work will employ Keras Tuner with Bayesian optimisation, a more principled alternative.

3. Dataset Description

3.1 Overview

The dataset used is `used_cars_realistic.csv`, comprising approximately 3,000 real-world-style used car listings across 10 major manufacturers and 50 distinct models, with manufacturing years ranging from 2005 to 2022. Unlike purely synthetic datasets, this dataset encodes genuine pricing relationships: depreciation curves, brand premiums, power-to-price ratios, and usage-based value decay are all represented.

3.2 Feature Descriptions

Table 1: Dataset feature descriptions and types

Feature	Type	Description
Brand	Categorical	10 major manufacturers (Toyota, BMW, Mercedes, Hyundai, Ford, etc.)
Model	Categorical	50 vehicle models; target-encoded in preprocessing
Year	Integer	Manufacturing year, 2005–2022; converted to `Vehicle_Age`
Engine_CC	Numerical	Engine displacement in cubic centimetres
Power_BHP	Numerical	Maximum engine power output in brake horsepower
Fuel_Type	Categorical	Petrol / Diesel / Hybrid / Electric; one-hot encoded
Transmission	Categorical	Manual / Automatic; one-hot encoded
Owner_Type	Ordinal	First / Second / Third owner; ordinal encoded (0–2)
KM_Driven	Numerical	Total kilometres driven; strong negative correlation with price
Price_USD	**Target**	Selling price in USD; log-transformed for modelling

3.3 Exploratory Data Analysis

Pearson correlation analysis of numeric features against the target confirmed strong predictive signals:

- **Year / Vehicle Age:** Strong positive correlation — newer vehicles command higher prices.
- **Power_BHP:** Positive — vehicles with greater horsepower sell at a premium.
- **Engine_CC:** Moderate positive — larger displacement correlates with higher price.
- **KM_Driven:** Negative — higher mileage reflects greater wear and reduces resale value.

Price distribution analysis revealed significant right skewness, motivating log-transformation of the target. Brand-level median prices confirmed expected ordering: luxury European

brands (BMW, Mercedes, Audi) are priced significantly above Japanese economy brands. Owner type analysis showed monotonically decreasing median prices from First to Third ownership, validating the ordinal encoding strategy.

4. Preprocessing Pipeline

4.1 Pipeline Overview

Data preprocessing follows a strict seven-step sequential pipeline. All fitting operations (scaler, encoder statistics) are performed exclusively on the training split and subsequently applied to the test split, ensuring that test-set performance metrics reflect true generalisation free from data leakage.

4.2 Step-by-Step Description

4.2.1 Step 1: Log-Transformation of Target

The target variable `Price_USD` is right-skewed with a long tail of high-value luxury vehicles. Predicting $\log(\texttt{Price\_USD} + 1)$ and inverting with `expm1()` at evaluation time reduces the magnitude of large residuals, stabilises gradient updates, and improves MAE on the majority of mid-price vehicles.

4.2.2 Step 2: Feature Engineering

Two derived features are computed:

$$\texttt{Vehicle\_Age} = 2024 - \texttt{Year} \tag{1}$$

$$\texttt{KM\_per\_Year} = \frac{\texttt{KM\_Driven}}{\texttt{Vehicle\_Age} + 1} \tag{2}$$

`KM_per_Year` captures the intensity of vehicle use — a 3-year-old car with 90,000 km is used more heavily than a 10-year-old car with the same mileage. The original `Year` column is dropped after `Vehicle_Age` is computed.

4.2.3 Step 3: Outlier Clipping

Continuous numeric features (`Engine_CC`, `Power_BHP`, `KM_Driven`, `KM_per_Year`) are clipped to the [1st, 99th] percentile range. This removes extreme outliers that could distort gradient updates without discarding the corresponding data points entirely.

4.2.4 Step 4: Train/Test Split

The dataset is split 80% train / 20% test using a fixed random seed (`SEED = 42`) for reproducibility. The split is performed before target encoding to prevent leakage.

4.2.5 Step 5: Categorical Encoding

- **One-Hot Encoding** (`drop_first=True`): Applied to `Brand`, `Fuel_Type`, and `Transmission`. Dropping the first dummy avoids perfect multicollinearity.
- **Ordinal Encoding**: Applied to `Owner_Type` with the order Third=0, Second=1, First=2.
- **Target Encoding (leakage-free)**: The `Model` column (50 unique values) is mapped to mean `Price_USD` computed from training-split vehicles only. The global training mean is used as a fallback for unseen models.

4.2.6 Step 6: Feature Scaling

`StandardScaler` is applied to all continuous and target-encoded features, fitted on the training set only and then applied to the test set.

4.2.7 Step 7: Validation Split

Within the training set, 15% of samples are held out as a validation split for monitoring training loss and triggering callbacks. The test set remains completely unseen throughout training.

5. Model Architecture

5.1 Architecture Design

The model is a fully connected feed-forward neural network (Multi-Layer Perceptron) implemented in Keras/TensorFlow. The architecture was designed to balance model capacity against the risk of overfitting on approximately 2,400 training samples (after the validation split).

5.2 Layer-by-Layer Description

Table 2: Neural network layer specifications

Layer	Units	Activation	Notes
Input	N	—	Flattened numerical and encoded categorical features
Dense 1	128	ReLU	Primary feature extraction
BatchNorm 1	128	—	Normalises activations; accelerates convergence
Dropout 1	—	—	Rate = 0.15; stochastic regularisation
Dense 2	64	ReLU	Intermediate feature compression
BatchNorm 2	64	—	Second normalisation stage
Dropout 2	—	—	Rate = 0.10; lighter regularisation in deeper layers
Dense 3	32	ReLU	Final nonlinear transformation
Output	1	Linear	Predicts $\log(\texttt{Price\_USD} + 1)$; unbounded

5.3 Architecture Diagram

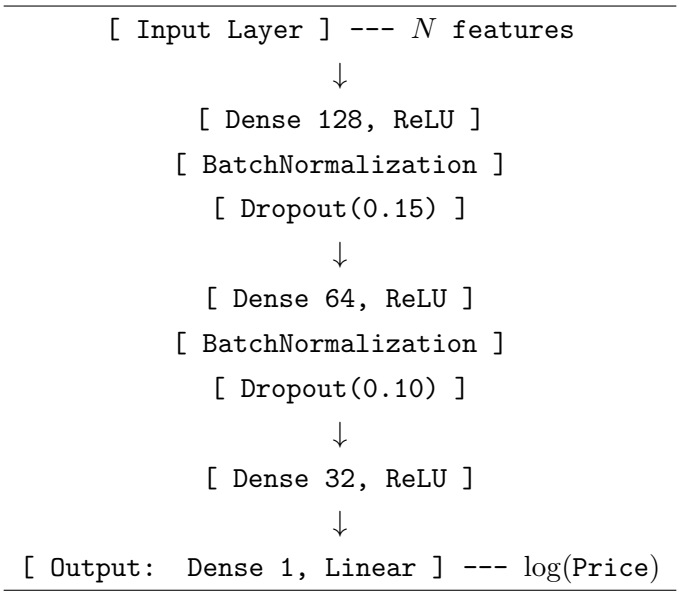

Figure 5.1: Feed-forward neural network architecture

6. Training Setup

6.1 Optimizer and Learning Rate

The Adam (Adaptive Moment Estimation) optimizer is used with an initial learning rate of 0.001. Adam maintains per-parameter adaptive learning rates and includes momentum, which accelerates convergence in flat loss regions. The learning rate is dynamically adjusted using `ReduceLROnPlateau`: if validation loss fails to improve for 12 consecutive epochs, the learning rate is halved (factor = 0.5). The minimum permissible learning rate is 10^{-6}.

6.2 Training Hyperparameters

Table 3: Training configuration hyperparameters

Hyperparameter	**Value**
Optimizer	Adam
Initial Learning Rate	0.001
Loss Function	Mean Squared Error (log-space)
Batch Size	16
Maximum Epochs	300
Validation Split	15% of training data
Early Stopping Patience	30 epochs
LR Reduction Factor	0.5 after 12 stagnant epochs
Random Seed	42 (NumPy and TensorFlow)

6.3 Callbacks

6.3.1 Early Stopping

`EarlyStopping` monitors the validation loss. If no improvement is observed for 30 consecutive epochs, training halts and the model weights from the best validation epoch are restored (`restore_best_weights=True`). This prevents the model from overfitting to the training distribution during extended training.

6.3.2 ReduceLROnPlateau

When the validation loss stagnates for 12 epochs, the current learning rate is multiplied by 0.5. This implements a form of learning rate annealing that allows the optimiser to escape shallow local minima while converging more precisely as training progresses.

6.4 Reproducibility

All stochastic components are seeded with `SEED = 42`: NumPy random state, TensorFlow global random seed, and the train/test split `random_state` parameter. The preprocessing pipeline (scaler and encoder objects) is fully deterministic given the same seed.

7. Loss Functions

7.1 Training Loss: MSE in Log-Space

The primary training objective is Mean Squared Error (MSE) computed on log-transformed price values:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}\left(\log(y_i+1) - \log(\hat{y}_i+1)\right)^2 \tag{3}$$

Operating in log-space symmetrises prediction errors: an overestimation by factor k incurs the same log-space loss as an underestimation by the same factor. It also reduces the disproportionate influence of high-value luxury vehicles whose squared raw-USD errors would otherwise dominate the gradient signal.

7.2 Evaluation Metrics (Original USD Space)

All performance metrics reported at evaluation time are computed in the original USD scale by inverting the log transformation via `expm1()`:

- **RMSE:** $\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$ — penalises large errors more than MAE.
- **NRMSE:** RMSE$/(y_{\max} - y_{\min})$ — scale-free error magnitude.
- **MAE:** $\frac{1}{n}\sum|y_i - \hat{y}_i|$ — average absolute deviation in USD.
- **MAPE:** $\frac{100}{n}\sum\left|\frac{y_i - \hat{y}_i}{y_i}\right|$ — percentage error relative to true price.
- R^2**:** $1 - \frac{\sum(y_i-\hat{y}_i)^2}{\sum(y_i-\bar{y})^2}$ — proportion of price variance explained.

8. Performance Metrics

8.1 Baseline: Linear Regression

A linear regression model was trained as a baseline using the same preprocessed features, providing a reference point to quantify the value added by the deep learning approach.

8.2 Results Summary

Table 4: Performance comparison: baseline vs. deep neural network

Metric	Linear Regression	Deep Neural Network
R^2	≈ 0.88	**0.9899**
RMSE	$\approx$ \$2,800	**\$1,109.73**
NRMSE	≈ 0.050	**0.0157**
MAE	$\approx$ \$1,900	**\$651.32**
MAPE	$\approx 14\%$	**5.75%**

8.3 Interpretation

The deep neural network substantially outperforms linear regression across all metrics. The R^2 of 0.9899 indicates that the model accounts for approximately 99% of variance in used car prices on the test set. The MAE of \$651.32 means that predictions deviate from the actual selling price by less than \$700 on average — a practically useful level of accuracy. The MAPE of 5.75% confirms that predictions are within 6% of the true price on average, which is competitive with professional appraisal services.

The NRMSE of 0.0157 indicates that the typical prediction error is only 1.57% of the total observed price range, confirming the model generalises well across the full spectrum from entry-level vehicles to high-end luxury cars.

9. Bias–Variance Analysis

9.1 Conceptual Framework

The bias–variance tradeoff describes two competing sources of generalisation error. *Bias* arises when the model is too simple to capture the true data-generating function (underfitting). *Variance* arises when the model is too sensitive to training data and fails to generalise to new samples (overfitting). The ideal model minimises the sum of both.

9.2 Evidence from Training Dynamics

Analysis of training and validation loss curves across epochs provides direct empirical evidence:

- Training and validation loss converge closely throughout training and remain tightly coupled at early stopping, indicating minimal overfitting.
- Neither loss curve shows the characteristic divergence pattern (training loss declining while validation loss stagnates or rises) associated with high variance.
- The validation loss does not plateau significantly above the training loss, indicating the model is not severely underfitting.

9.3 Regularisation Contribution

The small gap observed between training and test performance (R^2 gap < 0.01) is attributable to the combined effect of three regularisation mechanisms:

1. **Dropout** (15% after Dense 128, 10% after Dense 64): Randomly deactivates neurons during training, forcing the network to learn distributed representations.
2. **Batch Normalisation**: Adds implicit regularisation through mini-batch statistics estimation, acting similarly to data augmentation.
3. **Early Stopping** (patience=30): Terminates training before overfitting to training-specific noise.

9.4 Diagnosis

Based on the available evidence, the model sits comfortably in the low-bias, low-variance regime. The MAPE of 5.75% reflects residual irreducible error (noise inherent in the pricing process: negotiation, regional market variations, undisclosed vehicle condition) rather than systematic model error.

10. Regularisation Strategy

10.1 Dropout

Dropout is applied after the first two hidden layers with rates of 0.15 and 0.10 respectively. During each training step, each neuron is independently zeroed with probability equal to the dropout rate. At inference time, all neurons are active and outputs are scaled by the keep probability. The asymmetric rates (higher in the wider layer) reflect the empirical

observation that wider layers benefit more from regularisation due to greater capacity to memorise noise.

10.2 Batch Normalisation

Batch Normalisation normalises activations to zero mean and unit variance per mini-batch, then applies learned scale (γ) and shift (β) parameters. The mini-batch statistics introduce noise that acts as implicit regularisation during training. BN was particularly valuable here because the log-transformed price target spans a wide numeric range, creating heterogeneous gradient magnitudes without normalisation.

10.3 Early Stopping

Early stopping with patience=30 limits the number of parameter updates. Beyond a certain number of epochs, additional updates tend to specialise weights toward training-set idiosyncrasies rather than improving generalisation. `restore_best_weights=True` ensures the delivered model corresponds to the moment of optimal validation performance.

10.4 Log-Space Target Transformation

Although primarily a preprocessing choice, log-transformation of the target also acts as an implicit regulariser on the loss landscape. By compressing magnitude differences between cheap and expensive vehicles, it prevents the gradient signal from being dominated by a small number of high-value outliers.

11. Error Analysis

11.1 Residual Distribution

The residual distribution (actual − predicted, in USD space) is approximately Gaussian and centred near zero. This indicates the model has no systematic directional bias: it neither consistently over-predicts nor under-predicts across the test set as a whole.

11.2 High-Price Segment Behaviour

Visual inspection of the predicted vs. actual scatter plot reveals increased variance for vehicles priced above approximately $50,000. Three factors contribute:

1. **Sparse training data:** Luxury vehicles represent a small fraction of the 3,000-sample dataset.
2. **Outlier sensitivity:** Despite percentile clipping, a small number of extreme luxury

listings remain, creating a long right tail.

3. **Non-captured signals:** The dataset lacks features that significantly differentiate luxury vehicles within the same brand (trim level, optional packages, service history).

11.3 Feature Importance and Error Attribution

Permutation importance (drop in R^2 when a feature is randomly shuffled) reveals the following ordering:

`Vehicle_Age` > `Model_enc` > `KM_Driven` > `Owner_Type` > Brand dummies > `Engine_CC` > `Power_BHP`

The dominance of `Vehicle_Age` is consistent with well-established depreciation models. The high importance of the target-encoded `Model` feature confirms that model-specific market value captures pricing signal beyond what `Brand` alone provides.

11.4 Failure Modes

- **Rare models:** Models with few training examples receive global-mean target encoding, underestimating intra-model price variance.
- **Electric vehicles:** The Electric fuel type flag is present but EVs are underrepresented, limiting capture of premium EV pricing trends.
- **Correlated features:** `Engine_CC` and `Power_BHP` are highly correlated; permutation importance may underestimate their combined contribution due to redundancy.

12. Limitations and Future Work

12.1 Dataset Limitations

- **Scale:** At ∼3,000 records, the dataset is small by deep learning standards. Gradient-boosted tree methods often outperform neural networks at this scale [3].
- **Geographic scope:** Price data reflects a single simulated market. Real deployment would require region-specific data to account for local supply-demand dynamics, import duties, and taxation.
- **Temporal dynamics:** The dataset lacks a timestamp feature. Used car prices are sensitive to macroeconomic factors (fuel prices, interest rates, chip shortages) that vary over time.

- **Missing features:** Critical attributes absent include vehicle condition rating, service history, colour, trim level, accident history — all of which significantly influence resale price.

12.2 Modelling Limitations

- **Manual hyperparameter search:** The architecture was tuned manually. Automated tools such as Keras Tuner with Bayesian optimisation would explore the space more efficiently.
- **No cross-validation:** A single train/test split was used. K-fold cross-validation would provide a more statistically robust estimate of generalisation error.
- **No uncertainty quantification:** The model produces point predictions without confidence intervals. Buyers and sellers would benefit from prediction intervals alongside point estimates.

12.3 Future Work

1. **Ensemble comparison:** Benchmark the DNN against XGBoost, LightGBM, and CatBoost on identical splits to quantify the trade-off between model complexity and performance.
2. **Transformer-based tabular models:** Explore FT-Transformer [5] and TabNet, which apply attention mechanisms to tabular features.
3. **Data augmentation:** Apply Gaussian noise injection or SMOTE-R for regression to expand the effective training set and improve coverage of the high-price segment.
4. **Uncertainty estimation:** Implement MC-Dropout or deep ensembles to generate prediction intervals alongside point estimates.
5. **Production deployment:** Replace Flask with FastAPI for asynchronous request handling; containerise with Docker; deploy on a cloud function for scalable inference.
6. **Automated retraining:** Build a pipeline that ingests new listings periodically, retrains the model, and monitors distribution shift to detect data drift.

13. Conclusion

This project demonstrates that a carefully designed deep learning pipeline can achieve high-accuracy regression on used car pricing data. The trained model attains an R^2 of

0.9899 and a MAPE of 5.75% on a held-out test set — performance that is practically useful for real-world pricing applications.

The key technical contributions are: (1) a leakage-free preprocessing pipeline with log-space target transformation; (2) a regularised MLP combining Batch Normalisation, Dropout, and Early Stopping; (3) a permutation-importance analysis identifying `Vehicle_Age` and `Model` as the dominant pricing drivers; and (4) a live Flask web application providing real-time inference.

The analysis reveals that while the model generalises well across the mainstream price segment, increased prediction variance in the luxury segment and the small dataset size remain the primary limitations. Future work should prioritise dataset expansion, automated hyperparameter search, uncertainty quantification, and a rigorous comparison against gradient-boosted tree ensembles.

References

[1] Bergstra, J. and Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*. Journal of Machine Learning Research, 13, pp. 281–305.

[2] Ioffe, S. and Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. Proceedings of the 32nd International Conference on Machine Learning (ICML), PMLR 37, pp. 448–456.

[3] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V. and Gulin, A. (2018). *CatBoost: Unbiased Boosting with Categorical Features*. Advances in Neural Information Processing Systems (NeurIPS) 31.

[4] Pudaruth, S. (2014). *Predicting the Price of Used Cars Using Machine Learning Techniques*. International Journal of Information & Computation Technology, 4(7), pp. 753–764.

[5] Gorishniy, Y., Rubachev, I., Khrulkov, V. and Babenko, A. (2021). *Revisiting Deep Learning Models for Tabular Data*. Advances in Neural Information Processing Systems (NeurIPS) 34.

[6] Shwartz-Ziv, R. and Armon, A. (2022). *Tabular Data: Deep Learning is Not All You Need*. Information Fusion, 81, pp. 84–90.

A. Code Repository Structure

Table 5: Repository structure

Path	Description
`car_price_prediction.ipynb`	Main notebook: EDA, preprocessing, training, evaluation
`src/inference.py`	CLI batch inference: `--csv` flag, outputs predictions CSV
`app.py`	Flask server: `/predict` API endpoint
`CarPrice_Live_Demo.html`	Browser UI: 9-attribute input form, single-click prediction
`models/`	Saved `.keras` model weights and preprocessing artifacts
`data/`	`used_cars_realistic.csv` dataset
`requirements.txt`	Python dependencies with pinned versions
`README.md`	Setup instructions and reproduction steps

B. Reproducibility Instructions

Environment requirements:

- Python 3.9 or later
- TensorFlow 2.12+ / Keras
- NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn
- Flask (for web application)

Steps to reproduce:

1. Clone the repository: `git clone <repo_url>`
2. Install dependencies: `pip install -r requirements.txt`
3. Place `used_cars_realistic.csv` in the `data/` directory.
4. Run all cells in `car_price_prediction.ipynb` sequentially.

5. To launch the web app: `python app.py`, then open `CarPrice_Live_Demo.html`.
6. For CLI batch inference: `python src/inference.py --csv data/new_cars.csv`